

Resumen - Nivelación

Esta ruta de **nivelación** está diseñada para construir bases sólidas antes de avanzar a temas más complejos (LLMs, orquestación, agentes, RAG, etc.). La idea no es solo “aprender herramientas”, sino desarrollar tres capacidades que se vuelven críticas en todo el camino:

1. **Pensar como quien resuelve problemas** (pensamiento computacional).
 2. **Trabajar como en proyectos reales** (Git y control de versiones).
 3. **Entender y aplicar IA desde lo práctico** (fundamentos + Python aplicado).
-

1) Pensamiento computacional y lógica de programación

Objetivo

Entrenar una forma de pensar para:

- descomponer problemas grandes en partes manejables,
- identificar patrones,
- abstraer lo importante,
- y diseñar algoritmos claros.

Qué se aprende (más en detalle)

- **Fundamentos de la computación:**
 - Diferencia y relación entre **hardware** y **software**.
 - Qué hace un sistema operativo, qué hace un programa, y cómo “corre” un código.
- **Operaciones lógico-aritméticas:**
 - operadores, comparaciones, expresiones,
 - cómo se evalúan condiciones (verdadero/falso) y por qué importan.
- **Estructuras de control:**
 - **decisiones** (if/else): bifurcar caminos según condiciones.
 - **repeticiones** (loops): repetir acciones con criterio.
- **Modelado de soluciones:**
 - flujogramas (diagramas de flujo),
 - pseudocódigo,
 - lenguaje natural estructurado (pasos claros).

- **Arquitectura de Von Neumann (en concepto):**
 - entender “cómo piensa” una computadora en términos de memoria, CPU y ejecución.
- **Terminal / línea de comandos (intro):**
 - comandos básicos para moverte por carpetas, manipular archivos y comprender el entorno.
- **Buenas prácticas de resolución:**
 - planificar antes de programar,
 - detectar errores temprano,
 - explicar la solución (esto es clave para entrevistas y trabajo en equipo).

Resultado

Al final, vas a poder:

- explicar un problema y sus componentes,
 - proponer una solución paso a paso,
 - y traducir esa solución a código de manera ordenada.
-

2) Git y control de versiones (trabajar como en la vida real)

Objetivo

Aprender a usar Git para que todo lo que programes sea:

- reproducible,
- rastreable,
- y mejorable sin miedo a “romper” lo anterior.

Qué se aprende (más en detalle)

- **Instalación y configuración:** según sistema operativo.
- **Repositorio y versionado:**
 - iniciar un repo,
 - hacer commits con un mensaje claro,
 - comprender el historial.
- **Buenas prácticas:**
 - commits pequeños y coherentes,
 - mensajes descriptivos,
 - organizar cambios,
 - evitar errores típicos (por ejemplo, versionar archivos innecesarios).
- **Mentalidad de trabajo:**
 - guardar avances con frecuencia,
 - experimentar en ramas (cuando se avance a ese punto),

- documentar cambios.

Resultado

Vas a poder:

- versionar tus proyectos del curso,
 - volver atrás si algo sale mal,
 - y preparar terreno para trabajar colaborativamente.
-

3) Fundamentos de IA (conceptos) + Python aplicado (práctica)

Objetivo

Entender qué es IA hoy y cómo empezar a aplicarla con una base real en Python.

Fundamentos de IA (lo esencial)

- Qué es IA, ML (machine learning) y cómo se relacionan.
- Diferencia entre aprendizaje supervisado y no supervisado (nivel introductorio).
- Conceptos de datos, variables, patrones, generalización y limitaciones.
- Qué significa “aplicar IA”: no es magia, es diseño + datos + evaluación.

Python aplicado a IA

- **Python base:**
 - variables, tipos, strings,
 - condicionales, loops, funciones.
- **Trabajo en Colab:**
 - entorno de notebooks,
 - ejecución por celdas,
 - buenas prácticas mínimas de orden.
- **Estructuras de datos:**
 - listas, diccionarios y su uso para organizar información.
- **Datos con librerías:**
 - Pandas (tablas, lectura de CSV, transformaciones),
 - NumPy (arreglos y operaciones).
- **Integración con IA / APIs:**
 - consumo de APIs,
 - prototipos tipo chatbot,
 - automatización básica.

Resultado

Quedás con base suficiente para:

- seguir con LLMs y frameworks (LangChain / LangGraph),
 - entender mejor los ejemplos,
 - y no trabarte en lo “básico” cuando toque construir cosas más complejas.
-

4) Ruta recomendada (para sacarle más provecho)

1. **Pensamiento computacional** primero (estructura mental).
 2. **Git** desde el día 1 (todo lo que hagas, versionalo).
 3. **IA + Python aplicado** (conceptos + práctica) para entrar fuerte a lo que sigue.
-

Qué te habilita esta nivelación

Cuando avances a orquestación, agentes y RAG, vas a notar que esta base sirve para:

- no perderte al diseñar un flujo de herramientas,
- entender el razonamiento detrás de un agente,
- y mantener proyectos organizados y “presentables”.